

CAN REWARDING TASK-SWITCHING INCREASE COGNITIVE FLEXIBILITY?

Dylan M. Diaz

Departments of Computer Science and Psychology
California State University, San Bernardino
San Bernardino, CA, USA
dylan.diaz4811@coyote.csusb.edu

Yu-Chin Chiu

Department of Psychological Sciences
Purdue University
West Lafayette, IN, USA
yuchinchiu@purdue.edu

ABSTRACT

Cognitive control enables us to flexibly switch between tasks, although doing so is effortful and costly: laboratory studies consistently show that switch trials produce slower responses and more errors than repeat trials. When given the freedom to choose, people also tend to repeat the task they have just performed instead of switching to a different one. Braem (2017) found that participants rewarded for task switches showed greater subsequent voluntary switching than participants rewarded for task repetitions. One interpretation is that participants learned an association between reward and switching, making them more willing to switch later. However, because that study compared separate groups of participants, it remains unclear whether participants learned a general preference for switching or a specific association between reward and a particular control operation, and a later conceptual replication obtained the effect only when participants were explicitly told the reinforcement schedule (Held et al., 2024). Here, we tested whether rewards can shape specific cognitive control operations, switching versus repeating, by manipulating reward contingencies within individuals. In Experiment 1, using behavioral measures, we found that participants learned the specific association between individual items and the rewarded control operation. Surprisingly, this learning produced larger switch costs for switch-rewarded items than for repeat-rewarded items. In Experiment 2, we extended this work to the neural level using EEG and time-resolved multivariate pattern analysis (MVPA). Decoding separates the switch- and repeat-rewarded conditions from scalp activity beginning at 132 ms after item onset. A classifier trained on blocks where item and context were confounded still separated the conditions in a neutral block where both item types appeared in a shared context, showing that the decoding effect cannot be explained by block-level reward context alone. Never-rewarded diagnostic items came to reflect the reward state of the list they occupied later in the epoch and more weakly. Decoding did not reach significance for held-out images. Together, these results suggest that reward-control associations can be learned at the level of individual items within participants.

Contents

1	Introduction and Purpose of Document	4
2	Experimental Design	4
2.1	Participants	4
2.2	Task and trial structure	5
2.3	Reward conditions and items	5

2.4	Trigger codes	6
2.5	Blocks, lists, and the built-in dissociation	6
3	EEG Acquisition and Preprocessing	7
3.1	Stage 1: filtering and first ICA pass	7
3.2	Stage 2: eye components, bad channels, and second ICA pass	7
3.3	Component selection criteria	7
3.4	Stage 3: crop, re-reference, epoch, and flag	8
3.5	Bin definitions	8
4	Epoch Metadata, Alignment, and Verification	9
4.1	Per-epoch metadata	9
4.2	Alignment and the merge-seam case	9
4.3	Verification before writing and at load time	9
5	Decoding Framework	10
5.1	Shared decoding procedure	10
5.2	Cross-validation versus generalization	10
5.3	Analysis 1: decoding reward history	11
5.4	Analysis 2: neutral-context transfer	12
5.5	Analysis 3: transfer to unrewarded probes	13
5.6	Analysis 4: generalization to unseen pictures	14
6	Supporting Analyses	15
6.1	Event-related potentials	15
6.2	Temporal generalization	16
6.3	Haufe-transformed topographies	17
6.4	Brain–behavior relationship	18
7	Reproducibility and File Map	19
8	Interpretive Summary	20
9	Running the Analysis Pipeline	20
9.1	MATLAB/EEGLAB and ERPLAB Pipeline	20
9.2	MNE-Python MVPA Pipeline	23
	Author Contributions	25
	Acknowledgments and AI Disclosure	25
	References	25

A Algorithms	27
B Results Summary	28
C Parameter Deviations by Analysis	29

1 Introduction and Purpose of Document

This document is a methods and analysis reference for the EEG component of the ISSC_wCue project. Its main purpose is to serve as a detailed description of the results found as well as documentation such that a new lab member should be able to reconstruct what was collected, how the raw EEG became clean epoched data, and exactly what each analysis computes and why. Supplemental material includes the MATLAB and EEGLAB preprocessing scripts, the Python and MNE decoding scripts, the bin descriptor files, and the per-subject metadata.

The central question is whether reward can shape a specific cognitive-control operation, switching versus repeating, at the level of individual stimuli and within a single participant. The behavioral experiments established that participants learn item-specific reward-control associations. The EEG experiment asks whether those associations are visible in the neural response to an item, when they emerge in time, and what they are specific to.

The main findings are:

- Reward history was decodable from EEG beginning at 132 ms after item onset.
- The neural signal generalized to a neutral block, supporting an item-specific reward association beyond block context alone.
- Never-rewarded diagnostic items came to reflect the reward state of their list, with a later and weaker effect.
- Cross-image generalization to unseen pictures was not detected.

Related Work. Task switching typically carries a behavioral cost, with switch trials producing slower responses and more errors than task repetitions (Kiesel et al., 2010).

Braem (2017) rewarded task alternations disproportionately in one group and task repetitions in another during a cued phase, then measured voluntary switching in an unrewarded free phase. The group rewarded for alternating went on to switch more often, and that increased switching came at a cost, with the repetition group showing the smaller task-rule congruency effect, interpreted as better task shielding. The design was between-subjects and blocked, with coin-flip instructions on the free phase.

Held et al. (2024) attempted a conceptual replication with free-choice trials interspersed among cued trials and a double-registration procedure that separates task choice from task execution. Their first experiment found no effect of the reward manipulation on voluntary switch rate; choices were dominated instead by a preference for the easier of the two tasks. The effect appeared only in a second experiment in which participants were explicitly informed of their reinforcement schedule, and there it was not robust to every preregistered outlier criterion.

Neither result fully resolves whether the association is general or specific, and this shapes the core of the present design. A between-subjects comparison cannot separate a general disposition to switch from a learned association, so the manipulation is moved within participants. And because the contingency in both prior studies applied globally to trial transitions, where it proved hard to learn, here it is attached to individual items, which gives the analysis a way to ask about the specificity of reward.

2 Experimental Design

2.1 Participants

Seventy Purdue University undergraduates were recruited, yielding a final sample of $N = 58$ after exclusions. All participants provided informed consent under a protocol approved by the Purdue University Institutional Review Board.

Twelve participants were excluded. Seven (S19, S30, S32, S39, S44, S62, S63) were dropped for low behavioral performance. Five (S34, S50, S61, S64, S67) were dropped for insufficient trials

per bin (see Section 3.5), fewer than 40 surviving in at least one bin after bad-channel removal and artifact rejection.

2.2 Task and trial structure

Participants performed a cued task-switching paradigm. On each trial an image (the item) appeared first, after which a task cue, a red or blue colored rectangle framing the image, indicated which of two classification tasks to perform: whether the pictured object was living or nonliving (animacy task), or a size judgement against a fixed reference object e.g. smaller or larger than a shoebox (size task). Cue color mapped to task. The task was always answered on a fixed pair of keys (animacy on c/m, size on d/k), and the assignment of category to key within each pair was counterbalanced across participants. The image and cue remained on screen until a response or until a deadline of 1500 ms from cue onset; no recorded response exceeded 1488.8 ms, and 986 trials in blocks 1 through 5 recorded no response. Feedback, reward amount for reward-bearing items or accuracy feedback for diagnostic items, was shown after the response, followed by an intertrial interval. All values reported in this section were computed from the behavioral record (E3_N70.csv).

2.3 Reward conditions and items

Each participant viewed 12 unique images, drawn randomly per participant. Items fall into three types, indexed at item onset by an EEG trigger: *rpCond* (repeat-rewarded, low switch reward, code 1), *swCond* (switch-rewarded, high switch reward, code 3), and *diagnostic* (never rewarded, code 2). There are four distinct images per type. Switch-rewarded items pay (score) out when the participant switches; repeat-rewarded items pay out when the participant repeats; diagnostic items pay nothing in either case and serve as a probe of context-level learning. Congruency is balanced 50/50 within every condition, and the four stimulus categories are balanced within each item set (Figure 1).

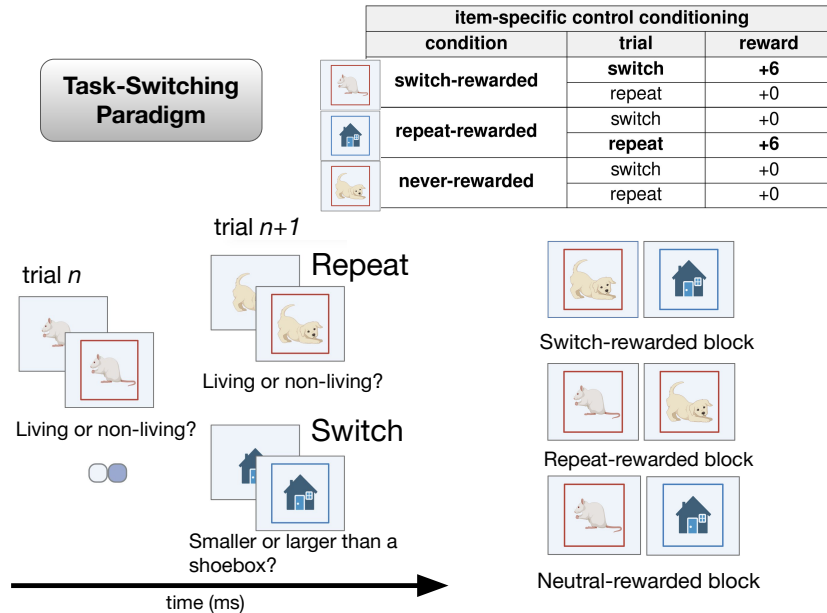


Figure 1: Task structure and item-specific conditioning manipulation. The contingency table illustrates the switch-rewarded, repeat-rewarded, and never-rewarded item conditions. Reward values shown in the figure are a schematic overview; the actual implemented task used probabilistic, speed-dependent rewards.

In the implemented task, a matching transition was scheduled for reward on 93% of trials, while non-matching transitions essentially never paid (0.0% for *swCond* items on repeat trials and 0.6% for *rpCond* items on switch trials). When delivered, reward was 6 points at baseline and approximately

8 points when the response beat the participant’s RT threshold. Rewards were delivered only on correct responses. Transitions were balanced 50/50 within each item type.

2.4 Trigger codes

Table 1 lists the marker codes written to the EEG stream. Item type is carried by codes 1 to 3 at item onset. The task cue codes carry two dimensions at once: 4 and 5 are switch cues (to animacy and to size respectively), 6 and 7 are repeat cues. Codes 13 to 16 are the four valid response keys. Code 17 marks any other key and is not included in the bin definitions described in Section 3.5.

Table 1: Marker codes.

Code	Meaning
1, 2, 3	item onset: <code>rpCond</code> (lowSR), <code>diagnostic</code> (noR), <code>swCond</code> (highSR)
4, 5	switch cue, to animacy and to size
6, 7	repeat cue, animacy and size
8	voluntary task cue (blocks 6–7 only)
9	block onset
10, 11, 12	feedback onset: incorrect, correct, too slow or no response
13–16	response keys
17	any other key

2.5 Blocks, lists, and the built-in dissociation

The cued phase comprises five blocks of 96 trials organized into three lists (episodes), shown in Table 2, beginning at trial ids 1, 97, 193, 289, and 385. The switch-conditioned list contains `swCond` and diagnostic items and carries a high block-level switch-reward probability; the repeat-conditioned list contains `rpCond` and diagnostic items and a low probability; the neutral list contains both inducer types and no diagnostic items. The layout is mirrored so that each item’s mean block position is 3.0, canceling linear time-on-task drift.

Two further blocks follow (6 and 7, beginning at trial ids 481 and 577). These are hybrid blocks containing both inducer types and no diagnostic items, and they intersperse voluntary-choice trials (task cue code 8, one third of trials) among instructed trials, which makes them the analog of the interspersed free-choice procedure in Held et al. (2024). **Every analysis in this document is restricted to blocks 1 through 5**; the voluntary blocks are cropped away at stage 3 of the EEG preprocessing pipeline (see Section 3) and are not examined here.

Table 2: Block structure of the cued phase.

Block	List (episode)	Block context (bkSRProb)	Item types present
1	SC (episode 1)	high	<code>swCond</code> + diagnostic
2	RC (episode 2)	low	<code>rpCond</code> + diagnostic
3	Neutral (episode 3)	neutral	<code>rpCond</code> + <code>swCond</code>
4	RC (mirror of 2)	low	<code>rpCond</code> + diagnostic
5	SC (mirror of 1)	high	<code>swCond</code> + diagnostic

Across blocks 1, 2, 4, and 5, an item’s contingency is aligned with its block’s reward context. `swCond` appears only in high-context blocks and `rpCond` only in low-context blocks. As shown in Table 2, block 3 is the exception as both inducer types appear there in a shared neutral context, allowing the item to be separated from its context. The diagnostic item allows the opposite separation. It is physically the same four images in both lists, so the list can be separated from the image. Analysis 2 holds context constant to isolate the item (Section 5.4), whereas Analysis 3 holds the image constant to isolate the list (Section 5.5).

3 EEG Acquisition and Preprocessing

Continuous EEG was recorded with a BrainVision system, referenced online to Cz. Sixty-three scalp channels are read from the raw files; the EOG channel is dropped at load time, and Cz is reconstructed as a data channel before average-referencing, giving 64 channels. Preprocessing runs in MATLAB via EEGLAB (Delorme & Makeig, 2004) with ERPLAB (Lopez-Calderon & Luck, 2014) across three numbered stages, implemented in `step1_ICAv1.m`, `step2_humanPP.m`, and the two stage-3 variants named in Section 3.4. Stages 1 and 2 perform two-pass ICA cleaning with human inspection between passes; stage 3 produces the epoched, artifact-clean datasets and the per-epoch metadata.

3.1 Stage 1: filtering and first ICA pass

Script: `step1_ICAv1.m`. Raw `.vhdr` recordings are resampled to 250 Hz and band-pass filtered from 0.2 to 20 Hz. Gaps longer than 4 s between events are removed with ERPLAB’s time-segment tool, keeping a 200 ms buffer before and 1000 ms after the flanking event codes. This dataset is saved as `S<sid>_preICA.set` and is the version all later stages return to for full-resolution data.

For the ICA decomposition only, a copy is downsampled to 100 Hz and high-passed at 2 Hz. Extended infomax ICA (Lee et al., 1999) is run on that copy, and the components are labeled with ICLabel (Pion-Tonachini et al., 2019), yielding `S<sid>_ICAv1.set`.

3.2 Stage 2: eye components, bad channels, and second ICA pass

Script: `step2_humanPP.m`, run in two chunks separated by a `return`, with manual inspection in between.

The first chunk removes the blink and saccade components identified in `ICAv1`, which are recorded per subject as an explicit index list rather than selected automatically. Bad channels are then detected on the cleaned data by normalized kurtosis (threshold 5), with Fp1 and Fp2 protected from rejection because frontal channels carry residual ocular activity that kurtosis flags even when the channel is good. A second ICA is computed on the surviving channels of the full-resolution `preICA` data, again downsampled to 100 Hz and high-passed at 2 Hz for the decomposition. Its weights are transferred back onto the full-channel dataset, which is relabeled with ICLabel and saved as `S<sid>_postICA.set`. The script stops here.

Each `postICA` dataset is then inspected by eye and any residual ocular components are listed for the second chunk, which removes them, interpolates the channels held out of the second ICA, and writes `S<sid>_ICApruned.set`. Both passes use the selection criteria described in Section 3.3.

One participant (S60) was recorded across two files that were later merged, producing a mid-block recording seam that the epoch-alignment code handles explicitly (Section 4.2).

3.3 Component selection criteria

Component removal is a manual decision at both ICA passes following a fixed procedure:

1. run the decomposition and label the components with ICLabel;
2. note every component assigned to the eye category, together with its classification probability;
3. scroll the component activation alongside the continuous EEG and confirm that its deflections coincide with those in Fpz and the neighboring frontal channels, the direct signature of a blink;
4. confirm the scalp topography shows frontal concentration;
5. confirm the power spectrum has no peak near 10 Hz, since an ocular component carries low-frequency power without the alpha peak a posterior cortical component would show.

A component is removed only when the topography, spectrum, and activity in the frontal channels all support the same conclusion. ICLabel probability is used as one part of this decision. Components labeled as eye-related with greater than 90% probability are still checked against the other criteria, because some high-probability components can reflect frontal cortical activity or single-channel noise. When the ICLabel probability is lower, especially below about 80%, more weight is given to the topography, spectrum, and frontal channel activity. For highly ambiguous components, around 60% and below, the component is usually retained.

3.4 Stage 3: crop, re-reference, epoch, and flag

Stage 3 exists in two variants. `step3_createERP.m` uses `binlister_v2.txt` (six bins) and feeds the ERP analyses; `step3_createERP_mvpa_fixed_v5.m` uses `binlister_mvpa.txt` (nine bins, adding the diagnostic item) and additionally exports the per-epoch metadata described in Section 4. Both perform, per subject:

1. cropping to the cued phase, from the first block-onset marker (code 9) through the sample before the sixth, which both drops blocks 6 onward and keeps any practice events preceding block 1 out of the metadata sequence;
2. automatic bad-channel detection by normalized kurtosis (threshold 5), supplemented by a per-subject manual bad-channel map for cases the automatic step missed;
3. reconstruction of the online-reference channel Cz from stored template coordinates, followed by average re-referencing;
4. spherical interpolation of removed channels back to the full 64-channel montage (Perrin et al., 1989), with an assertion that the result is 64 channels at 250 Hz;
5. event-list creation and bin assignment via ERPLAB BINLISTER;
6. epoching from -200 to 800 ms relative to item onset;
7. moving-window peak-to-peak artifact detection ($100 \mu\text{V}$ threshold, 200 ms window, 100 ms step).

The ERP average is computed before flagged epochs are physically removed. ERPLAB's averager already excludes flagged epochs, so removing them afterward does not change the exported ERPs. The flagged epochs are then removed with `pop_rejepoch` so that the saved `S<sid>.preprocessed.set` contains only accepted epochs. This removal is needed because MNE-Python's EEGLAB reader does not preserve ERPLAB artifact flags; otherwise, flagged epochs would be treated as clean during decoding.

BINLISTER's time-locking pattern also requires both the current and immediately preceding trials to be correct. As a result, only current-correct, post-correct trials are epoched. The metadata export applies the same eligibility rule when aligning EEG epochs to the behavioral data (Section 4.2).

3.5 Bin definitions

Two descriptors are used. `binlister_v2.txt` defines six bins and feeds the ERP analyses. `binlister_mvpa.txt` repeats those six and adds bins 7 to 9 for the diagnostic item, which analysis 3 requires. Every bin follows the same pattern, shown here for bin 1:

$$\{11\}.\{1\}\{4;5;6;7\}\{13;14;15;16\}\{11\}$$

The marker meanings are listed in Table 1. The period marks the time-locking event, so epochs are centered on the item cue. Reading outward, the definition requires that the preceding feedback was correct (11), that the item cue is of the requested type, that a task cue follows, that the response was one of the four valid keys (13 to 16, excluding 17), and that the trial's own feedback was correct (11). Together the leading and trailing `{11}` restrict every bin to current-correct, post-correct trials.

Bins 1 and 2 are `rpCond` and `swCond` collapsed over cue type. Bins 3 to 6 split those two by switch cue (`{4;5}`) versus repeat cue (`{6;7}`). Bin 7 is the diagnostic item, and bins 8 and 9 split it the same way.

The ERP route derives seven further bins with a binoperator descriptor (`diff_wave.txt`). Bins 7 and 8 are the unweighted means of the switch and repeat cells, $(b_3 + b_5)/2$ and $(b_4 + b_6)/2$, and bin 9 is their difference. Bins 10 and 11 are the switch effect within each reward condition, $b_3 - b_4$ and $b_5 - b_6$, and bin 12 is the difference between those. Bin 13 is $b_2 - b_1$, the reward contrast. Note that these bin numbers are the ERP route's; they do not correspond to bins 7 to 9 of the MVPA descriptor, which are the diagnostic cells.

4 Epoch Metadata, Alignment, and Verification

4.1 Per-epoch metadata

The analyses use per-epoch metadata to select trials because the EEG event codes contain only the reward condition and do not identify the physical item, block, list, congruency, or behavioral outcome. Stage 3 writes one metadata row for each surviving epoch (`S<sid>_metadata.csv`) containing the trial ID, block, episode, trigger code, stimulus ID and image name, item type, item- and block-level switch-reward probabilities, task, congruency, accuracy, and reaction time. The Python loader attaches this table to the MNE epochs object. Inducer trials can then be selected with `itemType=='inducer'`, and the two reward conditions are identified by trigger code.

4.2 Alignment and the merge-seam case

The EEG and behavioral logs are separate records, so the metadata export in `step3_createERP_mvpa_fixed.v5.m` aligns them by trial order and verifies that alignment before writing anything. The Python loader `mvpa_io.py` does not create the alignment; it performs an independent verification when the exported EEG and metadata are loaded for analysis. Two sequences are compared: the ordered item-cue codes (1, 2, or 3) taken from the cropped continuous data, one per item event, and the `triggerCode1` column for that subject's blocks 1 through 5 in trial order. Item codes are used instead of `urevent` identifiers because ERPLAB reindexes event ids during event-list and epoch operations. Equal length and an element-for-element match give an exact alignment.

When the EEG holds fewer item cues than the task presented, as for the merged participant S60 (458 recorded against 480 presented), the export assumes one contiguous run of trials is missing and tests every possible start position, deleting that many consecutive behavioral entries and asking whether the remainder reproduces the EEG sequence exactly. Because the item cues form a three-symbol alphabet, several positions often pass. The recording seam breaks the tie: a boundary event not followed by a block-onset marker (code 9) and with item cues on both sides, the both-sides test excluding the recording-start and recording-end boundaries. The item cues preceding it give the gap start, which is accepted only if it is among the passing positions.

A second check runs after epoching. BINLISTER bins only current-correct, post-correct trials, so the export applies the same eligibility rule to the continuous data and compares the resulting expected item sequence against the epochs `pop_epochbin` produced. Alignment failures skip only the metadata for that subject; the ERP average and the clean dataset are still written.

4.3 Verification before writing and at load time

The MATLAB export script `step3_createERP_mvpa_fixed.v5.m` refuses to write metadata unless four conditions hold. The bin-eligible item sequence derived from the continuous data must equal the bin-eligible behavioral sequence. The epoch count must equal the number of eligible behavioral rows. Every epoch's time-zero item code must be readable. And the epoch-by-epoch item sequence must match the expected one. On any failure the script reports the mismatching epochs and skips only the metadata; the ERP average and the clean dataset are still written.

Reading the item code back out of an epoch needs care. ERPLAB stores epoch labels in the form `B1, 3 (S1)`, so parsing the leading number returns the bin (instead of the trigger). The helper

extracts the parenthesized source tag instead, and accepts a bare 1, 2, or 3 only when the label contains nothing else.

Metadata rows are written for artifact-accepted epochs only, carrying an `epoch_index` that numbers them 1 to N in saved order, since `pop_rejepoch` preserves order but rennumbers.

The Python loader `mvpa.io.py` then performs an independent load-time verification. It requires the number of saved EEG epochs to equal the number of metadata rows and re-derives each epoch’s item code from the MNE event label, cross-checking it against `triggerCode1`. Thus, alignment is constructed and checked in MATLAB, then checked again independently when the data enter the Python analysis pipeline.

5 Decoding Framework

5.1 Shared decoding procedure

All four analyses reuse one decoder with different train/test splits. The classifier is time-resolved and linear, applied independently at each time sample following standard time-resolved EEG decoding procedures (Grootswagers et al., 2017). The implementation uses MNE-Python (Gramfort et al., 2013): a scikit-learn pipeline (Pedregosa et al., 2011) of per-fold standardization followed by ℓ_2 -regularized logistic regression ($C = 1.0$, fixed, `liblinear` solver), wrapped in MNE’s `SlidingEstimator` and scored by the area under the ROC curve (AUC). Logistic regression was used as a simple linear classifier with ℓ_2 regularization and $C = 1.0$ fixed across analyses. It also provides continuous probabilistic scores directly, which are retained for the generalization analyses and later re-scoring. The fixed parameters shared across all four analyses are summarized in Table 3.

Pseudotrial averaging. Within each class, random groups of four same-class trials are averaged into a pseudotrial, raising signal-to-noise before classification (Scrivener et al., 2026). Grouping is disjoint within a repeat and any remainder smaller than four is discarded. The grouping is redrawn and the whole decode repeated 100 times, and the resulting AUC time courses are averaged, so no single arbitrary grouping drives the result. Because groups are formed independently within the training and test pools of the generalization analyses, no raw trial contributes to both sides of any train/test split.

Cluster-based permutation testing. At the group level, per-subject AUC time courses are tested against chance with a one-sample, one-sided cluster permutation test using 10,000 permutations, which controls the family-wise error rate across time by evaluating contiguous supra-threshold clusters against a permutation null (Maris & Oostenveld, 2007). The cluster-forming threshold is the one-tailed t critical value at $\alpha = .05$ with $N - 1$ degrees of freedom. Inference is at the group level throughout: within-subject folds and repeats are averaged into a single subject estimate and are never treated as independent observations.

Every script saves raw per-subject arrays, all repeats plus cross-validated predictions or held-out decision scores, so that any additional metric, confusion matrices, Cohen’s κ , or within-subject confidence intervals can be computed later without re-running. A provenance record listing parameters, subject list, and package versions is written alongside the outputs.

5.2 Cross-validation versus generalization

Whether an analysis cross-validates depends on whether its training and test trials come from the same pool. Analysis 1 decodes within a single pool of inducer trials and therefore uses stratified k -fold cross-validation. Analyses 2, 3, and 4 are generalization analyses: they train on one pool and test on a disjoint pool, a held-out block, the diagnostic items, or a held-out image, which is already an out-of-sample test, so no folds are needed for the generalization score itself. All four retain the 100 pseudotrial repeats and the group-level permutation test.

Table 3: Fixed parameters shared across analyses.

Parameter	Value
Classifier	ℓ_2 logistic regression, <code>liblinear</code>
Regularization C	1.0, fixed
Standardization	<code>StandardScaler</code> , fit within fold
Pseudotrial size	4 trials
Pseudotrial repeats	100
Metric	AUC
Chance level	0.5
Epoch window	−200 to 800 ms from item onset
Cluster α	.05, one-tailed
Permutations	10,000
Random seed	42

5.3 Analysis 1: decoding reward history

Script: `step1_model_a_decode.py`.

Analysis 1 decodes `rpCond` versus `swCond` from the inducer epochs at each time point, with stratified k -fold cross-validation inside each pseudotrial repeat. The number of folds is $\min(5, n_{\min})$, where $n_{\min}$ is the smaller class count in pseudotrials, and repeats yielding fewer than two pseudotrials in either class are skipped.

Training and testing pool inducer trials from all five blocks, not only the biased lists. Blocks 1, 2, 4, and 5 contribute trials in which `swCond` appears only in high-context blocks and `rpCond` only in low-context blocks, while block 3 contributes trials in which both share the neutral context. Analysis 1 asks only whether any separable signal exists, and is not intended to attribute that signal to an item-specific or a context-specific representation. Both attributions are deferred to analyses 2 through 4.

Decoding is at chance before and around item onset and rises to a significant, cluster-corrected effect with an onset at 132 ms, peaking at $\text{AUC} = 0.59$ at 200 ms and remaining above chance to the end of the epoch (132–796 ms, $p < .001$; $N = 58$; Figure 2). The flat pre-onset baseline indicates that the conditions are not decodable before item onset. Images were randomly assigned to reward conditions within each participant, reducing the likelihood that a consistent visual difference between conditions explains the group-level effect. However, because analysis 1 trains and tests on trials from the same images within a participant, image-specific information could still contribute to decoding. Generalization to unseen images is tested directly in analysis 4 (Section 5.6).

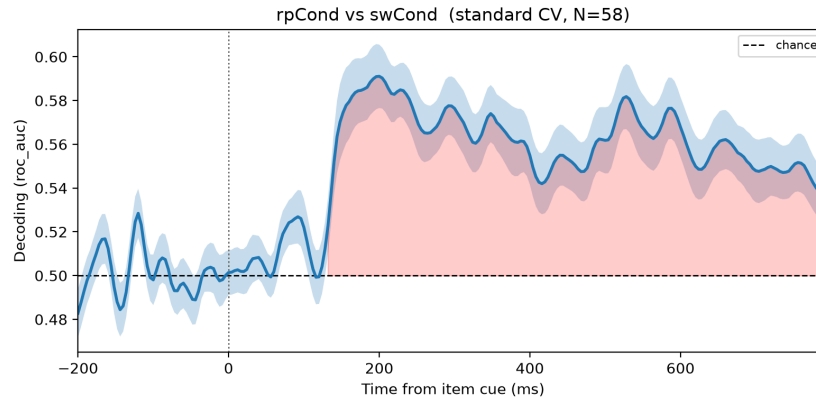


Figure 2: Analysis 1: time-resolved `rpCond` versus `swCond` decoding ($N = 58$). Group-mean AUC with SEM; the dashed line is chance, the dotted vertical line is item onset, and the shaded region marks the cluster-corrected significant window. Cluster 132–796 ms, $p < .001$; peak AUC = 0.59 at 200 ms.

5.4 Analysis 2: neutral-context transfer

Script: `step_2_model_a_neutral.py`.

Analysis 2 holds context constant in order to isolate the item. As described in Section 2.5 and summarized in Table 2, a classifier is trained to decode `rpCond` versus `swCond` on blocks 1, 2, 4, and 5, where item and context are entangled, and tested on block 3, the neutral list where both item types occur in a shared context. If the learned axis reflected block-level context it would fail in block 3, since no context difference exists there to read; if it reflects the item-specific association it should generalize. No cross-validation is used, because block 3 is entirely held out from training.

Decoding in the neutral block yields three significant clusters (Figure 3). The earliest and largest runs from 136 to 312 ms ($p = .002$) and contains the peak, AUC = 0.58 at 200 ms. Two later clusters follow, from 484 to 616 ms ($p = .010$) and from 656 to 732 ms ($p = .045$); the second of these is marginal, and no correction was applied across the three clusters.

Because the classifier is tested in block 3, where both inducer types occur in the same reward context, the result shows that decoding is not explained by block context alone. The signal may reflect the learned item association, but this analysis cannot separate that from information tied to the physical images, since the same images appear in the training blocks and in block 3. Analysis 4 tests generalization to unseen images and did not detect a reliable effect (Section 5.6).

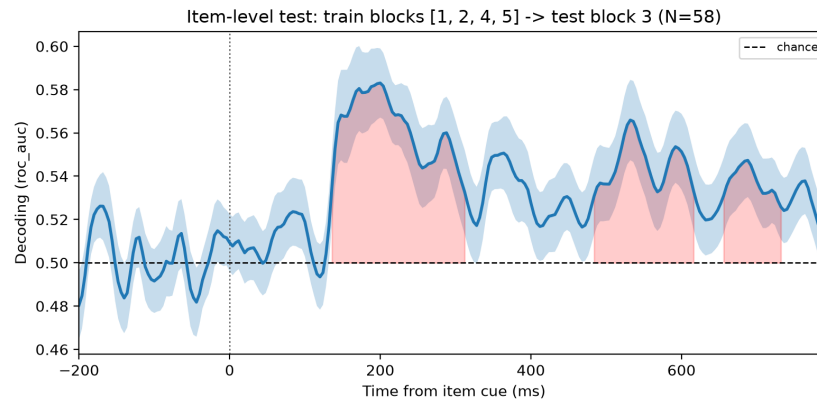


Figure 3: Analysis 2: generalization to the neutral block ($N = 58$). A decoder trained on blocks 1, 2, 4, and 5, where item and context are entangled, is tested on block 3, where both items occur in a neutral context. Significant clusters at 136–312 ms ($p = .002$), 484–616 ms ($p = .010$), and 656–732 ms ($p = .045$). Peak AUC = 0.58 at 200 ms.

Block 3 is a smaller test set than the training blocks, so this analysis has less sensitivity than Analysis 1. A subject is dropped unless both the training and block-3 sets contain at least 16 trials per class, ensuring at least four pseudotrials per class on each side. No subject was excluded by this criterion in this analysis. A secondary within-block-3 cross-validated decode is implemented and disabled by default, as the available block-3 trial counts make it substantially underpowered.

5.5 Analysis 3: transfer to unrewarded probes

Script: `step_3_model_b_diagnostic.py`.

Analysis 3 is the complement of Analysis 2 (Section 5.4) as it holds the image constant in order to isolate the list. The `rpCond`-versus-`swCond` axis is trained on all inducer trials, across blocks 1 through 5 as in analysis 1, and then applied to the diagnostic items, split by the list in which they occurred:

- `diag_SC`: episode 1, high context, blocks 1 and 5, alongside `swCond` inducers;
- `diag_RC`: episode 2, low context, blocks 2 and 4, alongside `rpCond` inducers.

The reported quantity is the AUC separating those two sets along the decoder's `swCond` axis, scored by the model's estimated probability of `swCond`. The logic is that diagnostic images carry no learned reward rule of their own, being the same four images in both lists. If the `swCond` axis separates the two diagnostic sets, the difference cannot be attributed to image identity because the same diagnostic images appear in both lists. Above-chance decoding therefore indicates that the diagnostic items are shifted in a list-consistent direction along the trained `swCond`–`rpCond` axis.

Because the same four diagnostic images appear in both lists, differences between `diag_SC` and `diag_RC` cannot be explained by the physical identity of the images. The diagnostic items are therefore useful for testing whether the learned list or context state transfers to stimuli that were never directly rewarded. Because each diagnostic item appears only within its assigned list, list membership and reward context are not separable in this analysis; Analysis 3 tests their combined influence on the diagnostic items.

A significant cluster appears late, from 420 to 644 ms, with a peak AUC of 0.54, $p = .002$ (Figure 4). Never-rewarded pictures therefore come to reflect the reward state of the context in which they appear, and they do so later and more weakly than the item-specific effect in analyses 1 and 2.

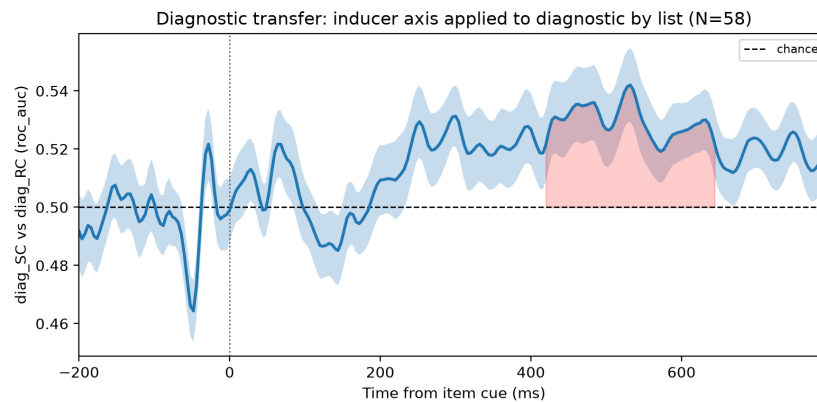


Figure 4: Analysis 3: transfer to unrewarded diagnostic probes. Significant cluster from 420 to 644 ms, peak $AUC = 0.54$, $p = .002$.

Because the diagnostic items are held out from training, the transfer score requires no cross-validation. A subject is dropped unless both the inducer and the diagnostic set contain at least 16 trials per class, ensuring at least four pseudotrials per class on each side. A separate within-inducer cross-validated decode was run inside `step_3_model_b_diagnostic.py` as a validity check on the training contrast. It uses the same contrast and trials as Analysis 1, but draws pseudotrial groupings from the per-subject random streams used by analyses 2 through 4 (Section 7) and applies a stricter inclusion rule of 16 trials per class. The two curves are expected to be similar but not numerically identical because the pseudotrial groupings are generated differently. All 58 subjects met the stricter rule. The within-inducer decode was above chance, indicating that the training axis contained reward-condition information (Figure 5). It was descriptive and was not used as an additional exclusion criterion.

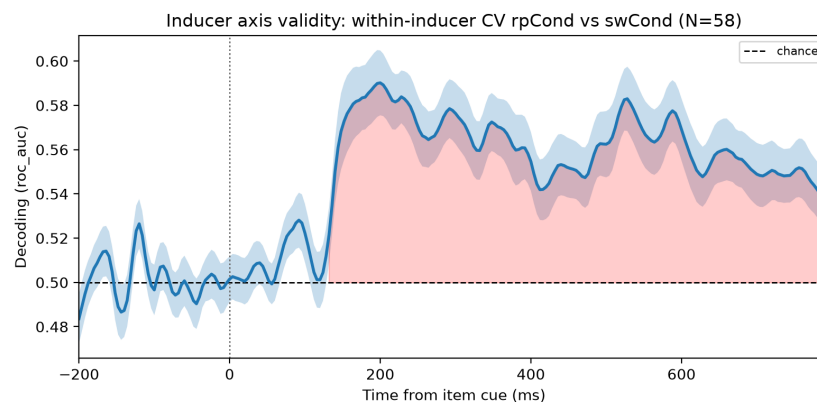


Figure 5: Within-inducer cross-validated decode ($N = 58$), run as a validity check on the axis that analysis 3 projects the diagnostic items onto.

5.6 Analysis 4: generalization to unseen pictures

Script: `step_4_model_c_loio.py`.

Analysis 4 tests whether the reward code is abstract across stimuli instead of being tied to specific images. Because analyses 1 through 3 train and test on the same four images per condition, above-chance decoding could in principle exploit image-specific visual features. Analysis 4 trains on three images per class and tests on the held-out image.

Why sixteen folds. Nothing in the design specifies which `swCond` image should be paired with which `rpCond` image, so instead of choosing four arbitrary pairings the procedure tests every possible held-out pairing: each of the four `swCond` images is crossed with each of the four `rpCond` images, giving $4 \times 4 = 16$ train/test splits. Each split trains on the remaining three images per condition and tests on the held-out pair. Crossing all combinations averages out the coin-flip character of selecting a single pairing and lowers the variance of the subject-level estimate.

A fold is used only if its training set contains at least 16 trials per class and its test set at least 8 per class, and a subject is dropped if fewer than four folds are usable. The 16 folds share trials and are therefore not treated as independent observations. They are averaged within each subject before the group-level statistical test.

Decoding did not reach significance at any time point (Figure 6).

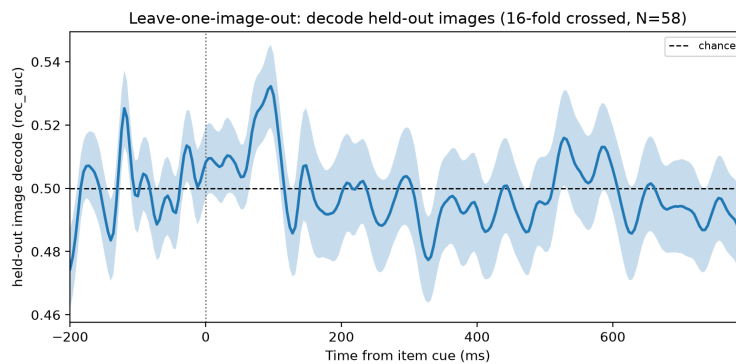


Figure 6: Analysis 4: generalization to held-out images. No significant cluster.

Interpreting the null. Analysis 4 did not detect generalization to unseen images. This does not show that the learned representation is image-specific. This analysis has the smallest test sets in the pipeline, with some folds containing as few as two pseudotrials per class from a single held-out image. The result therefore shows only that cross-image generalization was not detected with this analysis; it does not yet establish that cross-image generalization is absent.

6 Supporting Analyses

6.1 Event-related potentials

The ERP analyses run on the six-bin `binlister.v2.txt` output. Bins 1 and 2 are `rpCond` and `swCond`; bins 3 to 6 split those by switch versus repeat cue. Bins 7 to 13 are derived as described in Section 3.5. Waveforms are examined at Fz, Cz, Pz, and Oz. No additional filtering is applied at the ERP stage, since the 0.2 to 20 Hz band-pass from stage 1 already covers it.

The waveforms show visual components at Oz and a parietal P3 building from roughly 300 ms, while the two reward conditions largely overlap (Figure 7). Difference topographies are generated in 100 ms windows from 0 to 700 ms, with raw bins plotted on a $\pm 10 \mu\text{V}$ scale and difference bins on $\pm 1.5 \mu\text{V}$. On its own scale, the switch-rewarded minus repeat-rewarded difference (bin 13) stays small across the whole montage rather than only at the midline sites (Figure 8a).

The switch-minus-repeat difference (bin 9), computed from the same epochs and plotted on the same scale, produces a posterior positivity from about 400 ms (Figure 8b). In contrast, the reward-condition difference remains small across the scalp. This suggests that reward-related information may be distributed across channels and difficult to observe in single-channel mean amplitudes, motivating the use of multivariate decoding.

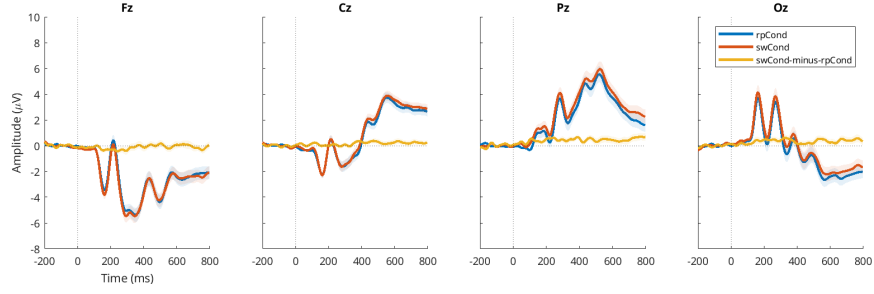


Figure 7: Stimulus-locked grand-average ERPs at Fz, Cz, Pz, and Oz ($N = 58$) with SEM shading. rpCond and swCond overlap and their difference wave is near zero throughout. The difference wave is drawn on the raw-ERP axis, so see Figure 8a for the same contrast at its own scale.

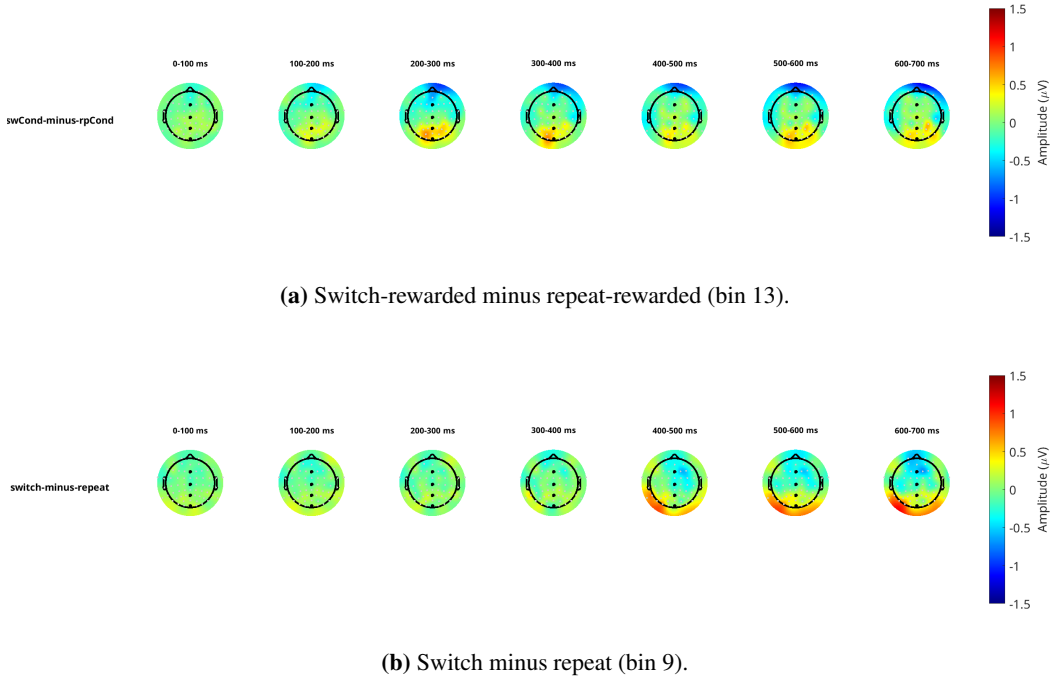


Figure 8: Difference topographies in 100 ms windows from 0 to 700 ms, both plotted at $\pm 1.5 \mu\text{V}$ ($N = 58$). The reward contrast (a) stays small across the montage, while the transition contrast (b), computed from the same epochs, produces a posterior positivity from about 400 ms.

6.2 Temporal generalization

Script: step_5_model_tg.v2.py.

The temporal generalization matrix trains a classifier at every time point and tests it at every other, yielding a train-time by test-time matrix whose structure distinguishes a representation that is sustained from one that evolves (King & Dehaene, 2014). It is run for two contrasts, which run sequentially and independently, each with its own cache and outputs.

The first is the Analysis 1 contrast (Section 5.3): rpCond versus swCond over all inducer trials, cross-validated. This analysis uses the full inducer-trial pool. Its diagonal can be compared with the time-resolved decoding result from Analysis 1. Because item contingency and block reward context are confounded in these trials, this matrix describes the time course of the overall decodable signal. It does not isolate the item association from block context. The second uses the Analysis 2 split

(Section 5.4): training on blocks 1, 2, 4, and 5 and testing on block 3. Because block 3 is already held out, this analysis does not use cross-validation for the test set. All available block 3 trials are used for testing.

Cross-validation for the first contrast splits over raw trials and wraps the entire matrix, so no trial is in train and test at any time pair. Fold count is derived per subject from the actual class counts rather than fixed, and every concrete split is checked before it is used.

The temporal generalization analyses include $N = 57$ participants rather than the $N = 58$ used in the primary MVPA analyses. One participant did not have enough surviving trials to form the minimum number of pseudotrials required for a usable temporal-generalization train/test split and was therefore excluded.

Figure 9 shows the Analysis 1 temporal-generalization matrix (rpCond versus swCond across all inducer trials). The context-controlled Analysis 2 matrix was run separately using the same framework but is not shown here.

Decimation. The data are decimated by a factor of 2, leaving roughly 125 time points. Because MNE's `decimate()` reduces the number of samples without applying a low-pass filter, the script first checks that the data have been sufficiently low-pass filtered before decimation. This helps prevent high-frequency activity from being misrepresented after the sampling rate is reduced. The low-pass value is set to 30 Hz, based on the PSD, because the EEGLAB filter setting is not carried into MNE.

Statistics. Unlike the four decoding analyses, the matrix is tested two-sided, with the threshold taken at $\alpha/2$, because the sign of an off-diagonal effect is meaningful: below-chance decoding can indicate that the learned pattern reverses across training and testing time points, while values near chance indicate no reliable separation. Above and below-chance clusters are kept separate and drawn differently (solid and dashed contours), and per-cluster records of sign, p , extent, and peak are saved.

Early in the epoch, decoding generalizes less across time, while later it generalizes more broadly. This suggests that the pattern captured by the decoder changes more early in the trial and becomes more stable later. The exact width of these regions should be interpreted cautiously because it can also be affected by filtering, noise, and variability in response timing.

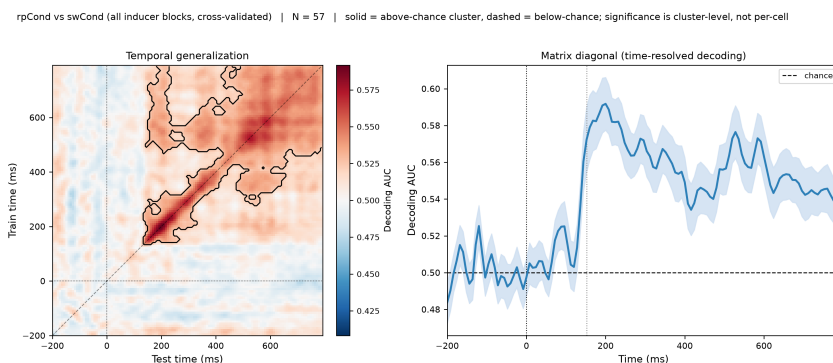


Figure 9: Temporal generalization for the Analysis 1 contrast, rpCond versus swCond across all inducer trials ($N = 57$). Training time is shown on one axis and testing time on the other; contours mark cluster-corrected significant regions, solid for above chance and dashed for below chance.

6.3 Haufe-transformed topographies

Script: `step_6_extract_patterns.v3.py`.

Classifier filter weights are not interpretable as neural sources, so weights are converted to forward activation patterns following Haufe et al. (2014) and plotted as scalp topographies. For each

epoch, EEG activity is first averaged across the 150 to 250 ms window, inside the significant Analysis 1 decoding cluster (Figure 2). The classifier and Haufe transformation are then applied to these window-averaged channel values, with patterns averaged across the 100 pseudotrial redraws. The pattern is dipolar, with posterior positivity for switch-rewarded items and a frontal negativity of comparable extent (Figure 10). Because the data are average-referenced, the positive and negative parts of the map should not automatically be treated as two separate neural sources. This topography comes from the *rpCond* versus *swCond* decoder and is separate from the switch-versus-repeat ERP contrast shown in Figure 8.

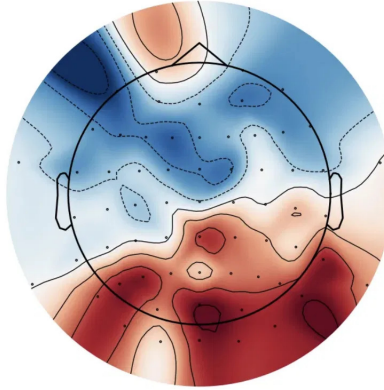


Figure 10: Haufe-transformed forward pattern for the *rpCond* versus *swCond* decoder, 150 to 250 ms ($N = 58$). Warm values are *swCond*-like and cool values *rpCond*-like; solid and dashed contours mark positive and negative values. Units are arbitrary, since forward patterns are defined up to scale.

6.4 Brain–behavior relationship

Script: `brain_behavior.py`.

Analysis 1 (Section 5.3) shows that the reward conditions are neurally decodable. This exploratory analysis asks whether individual differences in decoding strength are related to the behavioral reward-by-transition effect. If participants with stronger neural separation also show a larger behavioral interaction, this would link the neural signal to individual differences in the learned behavioral effect.

The two measures. The neural measure is each subject’s mean decoding AUC across a time window, read from the saved analysis 1 per-subject curves. The behavioral measure is the reward-by-transition interaction in reaction time, computed per subject (using the behavioral record `E3_N70.csv`) on cued-phase inducer trials that survive each subject’s own RT bounds:

$$\text{interaction} = (\text{RT}_{\text{sw,switch}} - \text{RT}_{\text{sw,repeat}}) - (\text{RT}_{\text{rp,switch}} - \text{RT}_{\text{rp,repeat}}).$$

All four cells must be present or the subject is dropped. Across the sample the interaction is positive, $M = 33.16$ ms, $SD = 44.26$, reproducing the behavioral finding that switch-rewarded items carried the larger switch cost.

Subjects are matched by identifier (not position) since `subject_ids.npy` is written in filename order, in which the underscore following a one-digit identifier sorts after a second digit (S10, S11, ..., S18, S1, S20, ..., S29, S2, S31), so positional alignment would silently mispair subjects. Spearman’s ρ is the primary test, with bootstrap 95% confidence intervals over 10,000 resamples and a leave-one-subject-out check for whether one participant strongly affects the result.

Result. The correlation was run over two windows as an exploratory analysis. The peak window is the 150–250 ms interval used for the Haufe patterns (Section 6.3); the full window is the entire group-level Analysis 1 decoding cluster. Both were run on the same $N = 58$ (Figure 11).

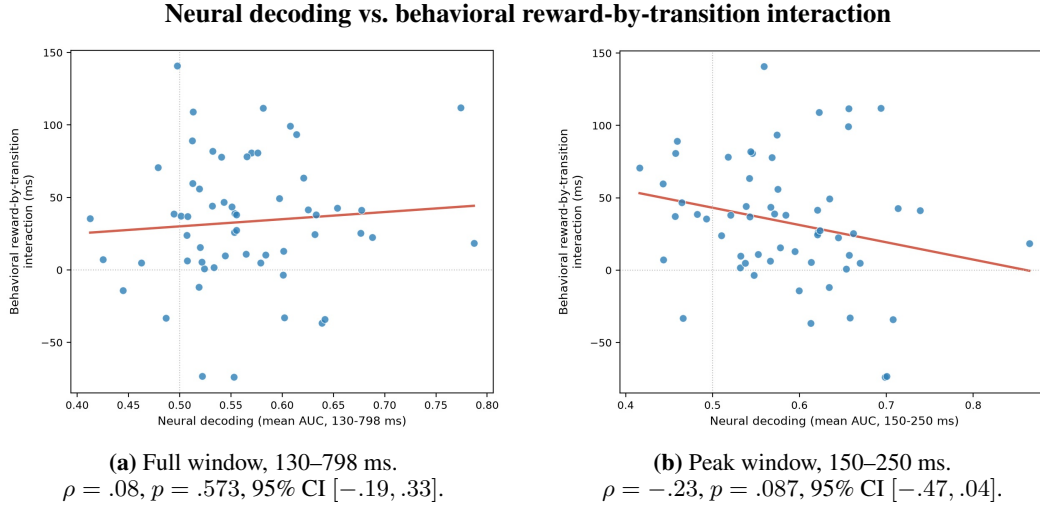


Figure 11: $N = 58$. Each point is one subject; the line is an OLS fit shown for visualization only. Neither correlation reaches significance, and the two differ in sign. Full statistics, including Pearson’s r and the leave-one-subject-out ranges, are in `brain_behavior.json` under each output directory.

Neither of the windows reach significance and the two differ in sign. The leave-one-subject-out ranges are narrow in both cases, so neither is driven by a single participant. The peak-window direction also runs against the hypothesis, since a negative ρ means subjects with stronger decoding showed a *smaller* behavioral interaction.

Both windows were selected from the group-level decoding result, not by searching for the strongest brain–behavior correlation. This avoids choosing a window based on the correlation being tested (Kriegeskorte et al., 2009; Vul et al., 2009). At $N = 58$ the test has roughly 60% power to detect $\rho = .30$ and 85% for $\rho = .40$.

Analyses 1 through 3 test group-level decoding and transfer, while this exploratory analysis tests individual differences. No reliable relationship between decoding strength and the behavioral reward-by-transition effect was detected in either window.

7 Reproducibility and File Map

The pipeline is reproducible when the same subjects, subject order, and random seed are used. The analysis scripts also save provenance files containing the main parameters and package versions used for each run. Analyses 2, 3, and 4 use a separate random-number stream for each subject, so a subject can be rerun without changing the others. Analysis 1 uses one shared random-number stream across subjects, so it will reproduce the same results in a full rerun with the same subject order, but restarting partway through the subject list may produce different random groupings for the remaining subjects.

Outputs for each analysis are written under `Results\MVPA_Models\<model>\`: full per-subject score arrays in `scores.full\`, held-out predictions or decision scores in `preds\`, grouped score matrices, subject-id vectors, a saved time axis, a provenance JSON, and the group figure. The raw per-repeat and per-prediction arrays are retained so that later requests for an alternative metric, within-subject confidence intervals, or a confusion matrix do not require recomputation.

The brain–behavior analysis reads the saved Analysis 1 subject-level decoding curves and writes its per-subject table, figure, and statistical provenance under `Results\Behavior\brain_behav_correlation\`, with separate outputs for the full-window and peak-AUC analyses in `full_window\` and `peak_auc\`, respectively. The provenance record includes the analysis window, included subjects, behavioral contrast, bootstrap settings, random seed, and correlation statistics. The main scripts and their roles are summarized in Table 4. For instructions on running the pipeline and the required script order, see Section 9.

Table 4: Pipeline components.

Component	Role
<code>step1_ICAv1.m</code>	Resample, filter, first ICA pass
<code>step2_humanPP.m</code>	Eye ICs, bad channels, second ICA, interpolation
<code>step3_createERP_mvpa_fixed.v5.m</code>	Stage-3 preprocessing, epoching, metadata export
<code>step3_createERP.m</code>	Stage-3 variant feeding the ERP analyses
<code>step4_ERP_difference.m</code>	Derived difference bins, group ERP matrix
<code>step5_ERP_plot.m</code>	Waveforms at Fz, Cz, Pz, Oz
<code>step5_Topo_plot.m</code>	Difference topographies
<code>binlister_mvpa.txt</code>	ERPLAB bin descriptor, 9 bins
<code>binlister_v2.txt</code>	ERPLAB bin descriptor, 6 bins (ERP route)
<code>diff_wave.txt</code>	Binoperator descriptor, derived bins 7–13
<code>mvpa_io.py</code>	Shared loader and alignment cross-check
<code>step1_model_a_decode.py</code>	Analysis 1: reward-history decoding
<code>step2_model_a_neutral.py</code>	Analysis 2: neutral-context transfer
<code>step3_model_b_diagnostic.py</code>	Analysis 3: transfer to diagnostic probes
<code>step4_model_c_loio.py</code>	Analysis 4: held-out-image generalization
<code>step5_model_tg.v2.py</code>	Temporal generalization analysis
<code>step6_extract_patterns.v3.py</code>	Hauke-transformed forward patterns
<code>brain_behavior.py</code>	Brain–behavior correlation and robustness checks
<code>S<sid>metadata.csv</code>	Per-epoch bridge to the design

8 Interpretive Summary

The four decoding analyses test different aspects of the reward-condition signal:

1. Reward learning produces a decodable neural signal. Brain activity distinguishes items linked to switching from items linked to repeating from 132 ms after item onset (Analysis 1; Section 5.3).
2. That signal cannot be explained by block-level context alone. It remains detectable in the neutral block, where switching and repeating are equally likely to be rewarded (Analysis 2; Section 5.4).
3. Broader context is also read, later in the epoch. Never-rewarded pictures come to reflect the reward state of the list they appear in, from 420 to 644 ms and at a smaller effect size (Analysis 3; Section 5.5).
4. Generalization across images was not detected. Decoding of held-out pictures did not reach significance (Analysis 4; Section 5.6), so this analysis does not provide evidence for cross-image generalization, but it also does not establish that the representation is image-specific.

Noting a few limitations, effects in this paradigm are modest in amplitude and distributed, hence the reliance on multivariate decoding as a complement to ERP difference waves, which are near-flat for these contrasts. Analysis 2 provides the main test of the item-level interpretation because both reward conditions are tested within the same neutral block context. Analysis 4 tests whether the signal generalizes to unseen pictures, and cross-image generalization was not detected.

9 Running the Analysis Pipeline

9.1 MATLAB/EEGLAB and ERPLAB Pipeline

Relevant folder: \EEG_Matlab

The EEG preprocessing and ERP pipeline is run in MATLAB using EEGLAB and ERPLAB. The required MATLAB toolboxes are listed in `Matlab_requirements.txt`. The archived analy-

sis folder also contains the EEGLAB version used for the final preprocessing. Before running the pipeline, confirm that the EEGLAB plugins ICLabel, ERPLAB, and bva-io are installed and available on the MATLAB path.

The preprocessing scripts use project-specific path variables near the top of each file. Before running a script on a new computer, update `rootDir` and any other path variables to point to the local project directory. In `step1_ICAv1.m`, also confirm that `Ref_path` points to the EEGLAB electrode-location file used by `pop_chanedit`. The recordings were acquired with Cz as the on-line reference.

A minimal working directory for the first preprocessing stages contains the raw BrainVision recordings and separate folders for the intermediate datasets:

```
ISSC_wCue/
  raw/
  preICA/
  pp/
```

The exact folder names can be changed, but the corresponding path variables in the MATLAB scripts must be updated before execution. Figure 12 summarizes the MATLAB preprocessing workflow and the point at which the ERP and MVPA pipelines diverge.

Step 1: initial filtering and first ICA. Start MATLAB and EEGLAB, then run `step1_ICAv1.m`. The script loads each BrainVision `.vhdr` recording, retains the 63 recorded scalp channels, reconstructs Cz, resamples the continuous data to 250 Hz, applies the 0.2–20 Hz band-pass filter, and removes long unused recording segments. The resulting continuous dataset is saved as `S<sid>_preICA.set`. A copy of this dataset is then downsampled to 100 Hz and high-pass filtered at 2 Hz for the first Extended Infomax ICA decomposition. ICLabel is applied and the resulting decomposition is saved as `S<sid>_ICAv1.set`.

The long-segment removal step uses `pop_erplabDeleteTimeSegments` with a 4000-ms threshold, a 200-ms buffer before the flanking event, and a 1000-ms buffer after it. The threshold should not be shortened without checking the task timing.

Before continuing, inspect one or more `preICA` datasets and confirm that the inserted boundary events occur only at the intended removed segments. If boundaries appear between ordinary trials, check the segment-removal settings before proceeding.

First ICA inspection. Open the resulting `S<sid>_ICAv1.set` datasets in EEGLAB/eStudio and use the ICA inspection tools to identify components associated with blinks, saccades, and other ocular activity. Record the component numbers for each subject. These selections are entered manually into `step2_humanPP.m`; they are not chosen automatically from the ICLabel classification.

If eStudio or ERPLAB functions are unavailable after starting EEGLAB, confirm that the ERPLAB plugin directory is on the MATLAB path before continuing.

Step 2: second ICA pass and manual cleaning. The final preprocessing pipeline uses `step2_humanPP.m`. Earlier versions of the project contained an alternative `step2_rmEyeIC.m` route, but that script is not part of the final pipeline documented here.

First, enter the ocular components identified from the first ICA inspection into `eyeIC_byNum`. The entries are positional by participant; for example, an entry of `[2 4]` for S1 indicates that components 2 and 4 should be removed for that participant.

Run the setup section of `step2_humanPP.m`, followed by Chunk 1. Chunk 1 removes the first-pass ocular components, detects bad channels using normalized kurtosis, computes a second ICA decomposition on the surviving channels, transfers the ICA weights back to the full-resolution data, applies ICLabel, and saves `S<sid>_postICA.set`. The script then stops at an explicit `return` statement.

Open the `postICA` datasets in EEGLAB/eStudio and inspect the second ICA decomposition for residual blink or saccade components. Enter any remaining ocular components in `eyeIC_v2`. If

MATLAB EEG Preprocessing Pipeline

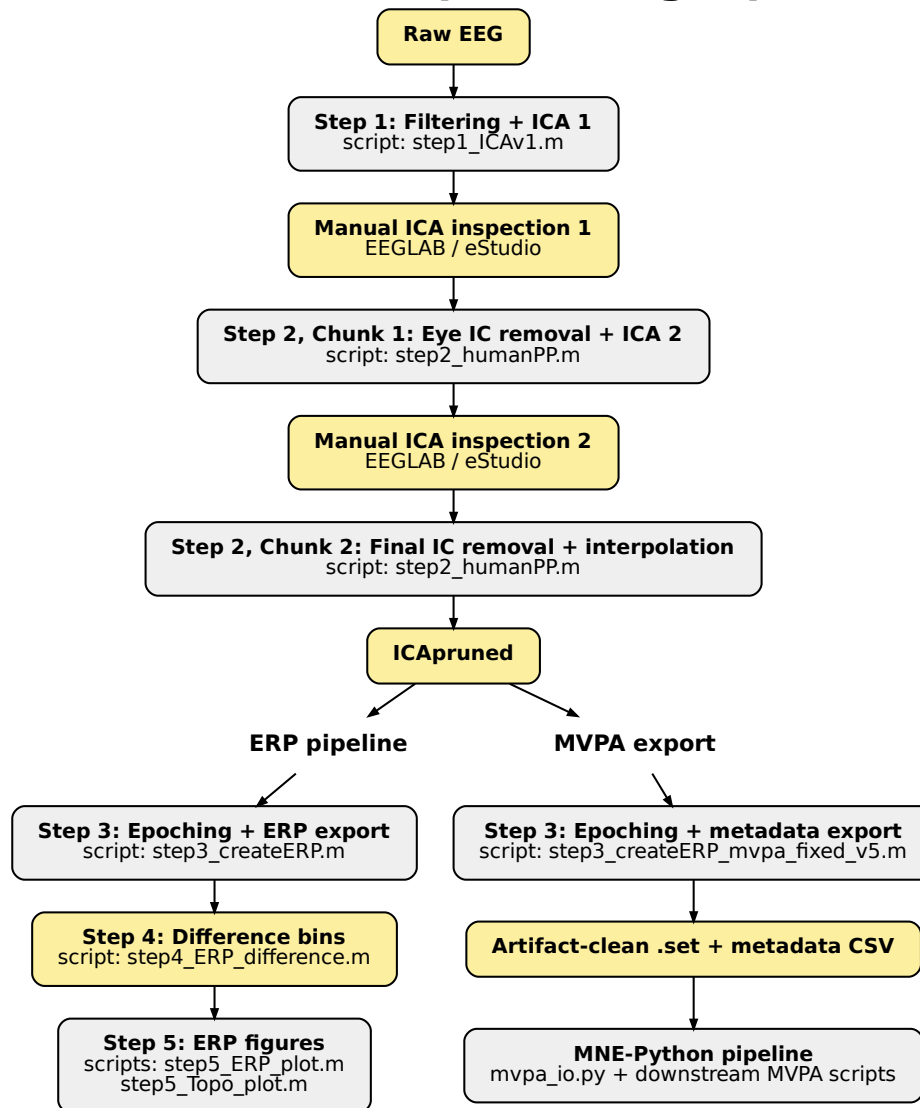


Figure 12: Workflow for the MATLAB/EEGLAB and ERPLAB preprocessing pipeline. The continuous EEG data pass through two ICA stages with manual component inspection between stages. After the final ICApruned datasets are created, the pipeline branches into the ERP analysis route and the MATLAB export used by the MNE-Python MVPA analyses.

MATLAB was restarted after Chunk 1, rerun the setup section before continuing so that the saved bad-channel information is reloaded.

Run Chunk 2 after the second manual inspection. This removes the residual ocular components, interpolates the channels excluded during the second ICA, and saves the final continuous cleaned dataset as `S<sid>.ICApruned.set`.

Special case: S60 recording merge. S60 was recorded in two parts. Both recordings are processed independently through Steps 1 and 2. After both have reached the ICApruned stage, run `S60_Combiner.m`. The script verifies that the two recordings have matching channel layouts, checks their chronological order, merges them with a boundary at the recording seam, backs up the

two original datasets, and saves the combined recording as `S60_ICApruned.set`. This merge must be completed before running Step 3 for S60.

Step 3: epoching and artifact rejection. After the ICApruned datasets have been created, the pipeline divides into an ERP route and an MVPA route.

For the ERP analyses, run `step3_createERP.m`. This script uses `binlister_v2.txt`, which defines six bins for `rpCond`, `swCond`, and their switch/repeat subdivisions. The script retains blocks 1–5, performs the final bad-channel checks and interpolation, restores Cz as a data channel, applies the common-average reference, creates the ERPLAB event list and bins, epochs the data from –200 to 800 ms around item onset, and applies the moving-window peak-to-peak artifact criterion.

For the MVPA analyses, run `step3_createERP_mvpa_fixed_v5.m`. This script instead uses `binlister_mvpa.txt`, which retains the six ERP bins and adds the diagnostic-item bins needed for the diagnostic-transfer analysis. It also reads the behavioral data from `E3.N70.xlsx`, aligns each surviving EEG epoch with its behavioral trial, and writes one `S<sid>metadata.csv` file per participant. These metadata files, together with the artifact-clean `S<sid>preprocessed.set` datasets, are the inputs to the MNE-Python MVPA pipeline.

The subject list used by the Stage-3 scripts should be checked before running the batch so that it matches the intended final sample.

Steps 4 and 5: ERP group files and figures. After the ERP version of Step 3 is complete, run `step4_ERP_difference.m`. This script loads the subject-level ERP files, computes the derived ERP bins defined in `diff_wave.txt`, and saves the combined group array as `gpERP_filt_allbins.mat`.

The group file is then used by the Step-5 plotting scripts. `step5_ERP_plot.m` produces the ERP waveform figures, and `step5_Topo_plot.m` produces the scalp-topography figures for the specified contrasts and time windows.

Manual checkpoints. The preprocessing pipeline is not fully automated. The ocular-component lists used in `step2_humanPP.m` were obtained by manual inspection of the first- and second-pass ICA decompositions (see Section 3.3 for manual-inspection procedure). The stored component indices reproduce the selections used for the present dataset and should not be reused for newly collected participants without inspecting their ICA decompositions.

9.2 MNE-Python MVPA Pipeline

Relevant folder: `\MVPA_Python`

The Python analyses use MNE-Python and scikit-learn. The final Python analyses were run in PyCharm 2026.2.1. The PyCharm IDE is not required; the scripts can also be run from a terminal or another Python environment such as Visual Studio Code, provided the packages in `MVPA_requirements.txt` are installed. Create a Python environment and install the required packages with:

```
pip install -r MVPA_requirements.txt
```

The supplied requirements file records the package versions used for the final analyses, including MNE-Python, NumPy, pandas, scikit-learn, SciPy, and Matplotlib.

Before running the decoding scripts, confirm that the artifact-clean epoched EEGLAB files and their matching metadata files are present. The shared loader, `mvpa_io.py`, expects the epoched `S<sid>preprocessed.set` files and one `S<sid>metadata.csv` file per subject. The project root and input directories are defined near the top of `mvpa_io.py` and should be changed if the project is moved to another computer.

The loader verifies that the number of metadata rows matches the number of EEG epochs and independently checks that the item code stored in each EEG epoch matches `triggerCode1` in the

MNE-Python MVPA Pipeline

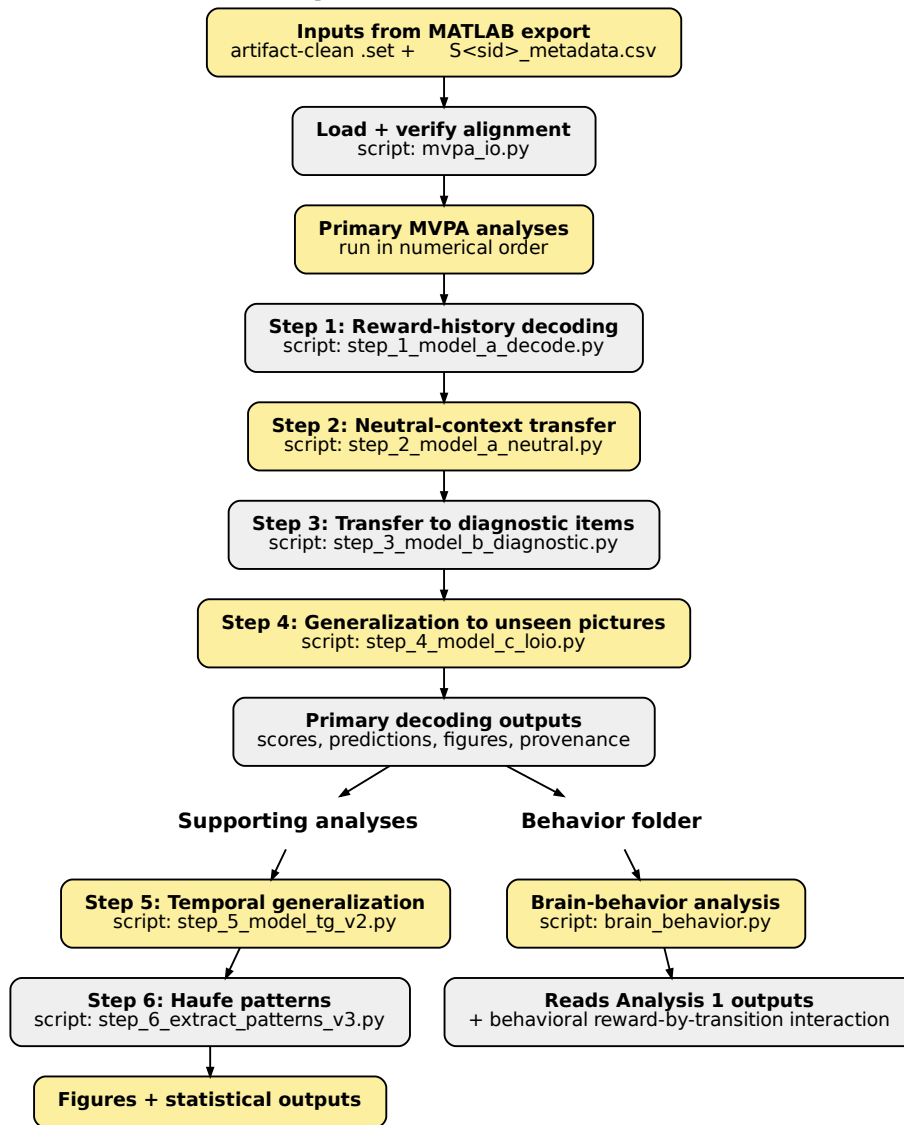


Figure 13: Workflow for the MNE-Python MVPA pipeline. Artifact-clean EEG epochs and trial-aligned metadata exported from MATLAB are verified by `mvpa_io.py` before entering the four primary decoding analyses. The temporal-generalization and Haufe-pattern analyses extend the primary MVPA results, while the brain-behavior analysis uses the saved Analysis 1 outputs together with the behavioral data.

metadata. Running `mvpa_io.py` directly can therefore be used as a basic loading and alignment check before running the analyses. The full Python analysis workflow is summarized in Figure 13.

Primary decoding analyses. Run the four primary analyses in numerical order:

```
python step_1_model_a_decode.py
python step_2_model_a_neutral.py
python step_3_model_b_diagnostic.py
python step_4_model_c_loio.py
```

Step 1 performs the cross-validated `rpCond` versus `swCond` decode. Step 2 trains on blocks 1, 2, 4, and 5 and tests generalization to the neutral block 3. Step 3 trains on inducer items and tests transfer to the never-rewarded diagnostic items. Step 4 performs the crossed leave-one-image-out analysis used to test generalization to unseen pictures.

Supporting multivariate analyses. After the four primary decoding analyses, run the temporal-generalization and Haufe-pattern scripts:

```
python step_5_model_tg_v2.py
python step_6_extract_patterns_v3.py
```

Step 5 computes temporal-generalization matrices by training the classifier at each time point and testing it at every other time point. It runs both the Analysis 1 `rpCond` versus `swCond` contrast and the context-controlled neutral-block contrast, saving the two analyses separately.

Step 6 computes the Haufe-transformed forward patterns used to interpret the spatial pattern contributing to decoding. It computes patterns for the main Analysis 1 contrast, the Analysis 2 training blocks, and block 3, and saves the resulting topographies and pattern-comparison outputs.

Brain-behavior analysis. The exploratory brain-behavior analysis is implemented in `brain_behavior.py`, stored in `\Behavior`. Run this analysis after Analysis 1 has been completed, because it reads the saved subject-level Analysis 1 decoding curves and relates each participant's decoding strength to the behavioral reward-by-transition interaction.

```
python brain_behavior.py
```

The script matches the neural and behavioral data by subject ID, and writes the per-subject analysis table, figure, and statistical provenance files.

Author Contributions

Y.-C.C.: conceptualization; literature review; methodology; investigation; resources; formal analysis; supervision; project administration; writing (review and editing). **D.M.D.:** software; formal analysis; data curation; visualization; writing (original draft). Both authors contributed to interpretation of the results. Data collection was complete before D.M.D. joined the project.

Acknowledgments and AI Disclosure

D.M.D. thanks the Purdue University Summer Research Opportunities Program (SROP) and the Big Ten Academic Alliance (BTAA) for funding and support during this study. D.M.D. also thanks Yu-Chin Chiu for her mentorship and guidance throughout the summer program. Claude Opus 5 was used to assist with code debugging, language refinement, file-format conversion and other utility scripting. All outputs were checked by D.M.D. against the source data. It was not used for study design, for the EEG preprocessing or decoding analyses, or for interpretation of results. D.M.D. takes full responsibility for the accuracy, analysis, and reporting of the methods and results presented in this document.

References

- Senne Braem. Conditioning task switching behavior. *Cognition*, 166:272–276, 2017. doi: 10.1016/j.cognition.2017.05.037.
- Arnaud Delorme and Scott Makeig. EEGLAB: An open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *Journal of Neuroscience Methods*, 134(1):9–21, 2004. doi: 10.1016/j.jneumeth.2003.10.009.

- Alexandre Gramfort, Martin Luessi, Eric Larson, Denis A. Engemann, Daniel Strohmeier, Christian Brodbeck, Roman Goj, Mainak Jas, Teon Brooks, Lauri Parkkonen, and Matti Hämäläinen. MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7:267, 2013. doi: 10.3389/fnins.2013.00267.
- Tijl Grootswagers, Susan G. Wardle, and Thomas A. Carlson. Decoding dynamic brain patterns from evoked responses: A tutorial on multivariate pattern analysis applied to time series neuroimaging data. *Journal of Cognitive Neuroscience*, 29(4):677–697, 2017. doi: 10.1162/jocn_a.01068.
- Stefan Haufe, Frank Meinecke, Kai Görden, Sven Dähne, John-Dylan Haynes, Benjamin Blankertz, and Felix Bießmann. On the interpretation of weight vectors of linear models in multivariate neuroimaging. *NeuroImage*, 87:96–110, 2014. doi: 10.1016/j.neuroimage.2013.10.067.
- Leslie K. Held, Luc Vermeulen, Ruth M. Krebs, Wim Notebaert, and Senne Braem. The effects of reinforcing task alternations on voluntary task selection. *Collabra: Psychology*, 10(1), 2024. doi: 10.1525/collabra.120329.
- Andrea Kiesel, Marco Steinhauser, Mike Wendt, Michael Falkenstein, Kerstin Jost, Andrea M. Philipp, and Iring Koch. Control and interference in task switching—a review. *Psychological Bulletin*, 136(5):849–874, 2010. doi: 10.1037/a0019842.
- Jean-Rémi King and Stanislas Dehaene. Characterizing the dynamics of mental representations: The temporal generalization method. *Trends in Cognitive Sciences*, 18(4):203–210, 2014. doi: 10.1016/j.tics.2014.01.002.
- Nikolaus Kriegeskorte, W. Kyle Simmons, Patrick S. F. Bellgowan, and Chris I. Baker. Circular analysis in systems neuroscience: The dangers of double dipping. *Nature Neuroscience*, 12(5): 535–540, 2009. doi: 10.1038/nn.2303.
- Te-Won Lee, Mark Girolami, and Terrence J. Sejnowski. Independent component analysis using an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural Computation*, 11(2):417–441, 1999. doi: 10.1162/089976699300016719.
- Javier Lopez-Calderon and Steven J. Luck. ERPLAB: An open-source toolbox for the analysis of event-related potentials. *Frontiers in Human Neuroscience*, 8:213, 2014. doi: 10.3389/fnhum.2014.00213.
- Eric Maris and Robert Oostenveld. Nonparametric statistical testing of EEG- and MEG-data. *Journal of Neuroscience Methods*, 164(1):177–190, 2007. doi: 10.1016/j.jneumeth.2007.03.024.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- F. Perrin, J. Pernier, O. Bertrand, and J. F. Echallier. Spherical splines for scalp potential and current density mapping. *Electroencephalography and Clinical Neurophysiology*, 72(2):184–187, 1989. doi: 10.1016/0013-4694(89)90180-6.
- Luca Pion-Tonachini, Ken Kreutz-Delgado, and Scott Makeig. ICLabel: An automated electroencephalographic independent component classifier, dataset, and website. *NeuroImage*, 198:181–197, 2019. doi: 10.1016/j.neuroimage.2019.05.026.
- Catriona L. Scrivener, Tijl Grootswagers, and Alexandra Woolgar. Optimising analysis choices for multivariate decoding: Creating pseudotrials using trial averaging and resampling. *European Journal of Neuroscience*, 64(2):e70601, 2026. doi: 10.1111/ejn.70601.
- Edward Vul, Christine Harris, Piotr Winkielman, and Harold Pashler. Puzzlingly high correlations in fMRI studies of emotion, personality, and social cognition. *Perspectives on Psychological Science*, 4(3):274–290, 2009. doi: 10.1111/j.1745-6924.2009.01125.x.

A Algorithms

Algorithm 1 Cross-validated decode (Analysis 1)

Require: Epochs X , labels y , repeats R , group size g

Ensure: AUC curve $a \in \mathbb{R}^T$

```

1: for  $r = 1$  to  $R$  do
2:   Form disjoint  $g$ -trial pseudotrials within each class
3:   Set  $k = \min(5, n_{\min})$ , where  $n_{\min}$  is the smaller pseudotrial class count
4:   if  $k < 2$  then
5:     Skip this repeat
6:   end if
7:   Split pseudotrials using stratified  $k$ -fold cross-validation
8:   for each fold do
9:     Fit the sliding estimator on the training pseudotrials
10:    Score the held-out pseudotrials at each time point
11:  end for
12:  Average AUC across folds
13: end for
14: return the mean AUC curve across valid repeats

```

Algorithm 2 Generalization decode (analyses 2, 3, 4)

Require: Epochs X , train mask, test mask, labels y , repeats R , group size g

Ensure: AUC curve $a \in \mathbb{R}^T$

```

1: Verify that train and test sets meet the minimum class counts specified for the analysis; otherwise
   skip the subject or fold
2: for  $r = 1$  to  $R$  do
3:   Form disjoint  $g$ -trial pseudotrials within each class, separately for train and test
4:   Fit the sliding estimator on the training pseudotrials
5:   Score the held-out pseudotrials, giving one AUC per time point
6:   Store predicted probabilities for later re-scoring
7: end for
8: return the mean curve over repeats

```

Algorithm 3 EEG–behavior alignment and verification

Require: Cropped continuous EEG events, behavioral trial table
Ensure: Trial-aligned metadata for each accepted EEG epoch

- 1: Extract the ordered EEG item-onset codes (1, 2, or 3)
- 2: Extract `triggerCode1` from behavioral blocks 1–5 in trial order
- 3: **if** EEG and behavioral sequences have equal length **then**
- 4: Require an element-by-element match
- 5: Use the behavioral rows in their original order
- 6: **else**
- 7: Compute the number of missing behavioral trials
- 8: **for** each possible start position of one contiguous missing run **do**
- 9: Remove that run from the behavioral sequence
- 10: **if** the remaining behavioral codes exactly match the EEG codes **then**
- 11: Store the start position as a possible alignment
- 12: **end if**
- 13: **end for**
- 14: **if** more than one start position matches **then**
- 15: Locate the recording seam from the EEG boundary event
- 16: Count the item-onset events preceding the seam
- 17: Use this count to identify the expected missing-run position
- 18: Require that this position is one of the possible alignments
- 19: **end if**
- 20: **end if**
- 21: Apply the same current-correct, post-correct eligibility rule used by BINLISTER
- 22: Retain the corresponding behavioral rows
- 23: Compare the expected eligible item-code sequence with the epoched EEG
- 24: Require the number of EEG epochs to equal the number of retained behavioral rows
- 25: Read the item-onset code at time zero from every EEG epoch
- 26: Require the epoch-by-epoch EEG codes to match the retained behavioral codes
- 27: **if** all checks pass **then**
- 28: Write one metadata row for each artifact-accepted EEG epoch
- 29: **else**
- 30: Report the alignment error and do not write metadata for that subject
- 31: **end if**

B Results Summary

Table 5: Group results across the four decoding analyses.

No.	Analysis	Window	Peak AUC	<i>N</i>
1	Decoding reward history	132–796 ms, $p < .001$	0.59	58
2	Neutral-context transfer	136–312 ms, $p = .002$ (+2 later clusters)	0.58	58
3	Transfer to unrewarded probes	420–644 ms, $p = .002$	0.54	58
4	Generalization to unseen pictures	no significant cluster	—	58

C Parameter Deviations by Analysis

Table 6: Additional parameters specific to individual analyses.

Analysis	Parameter	Value
1	maximum folds	5
2	minimum pseudotrials per class per side	4
3	minimum pseudotrials per class per side	4
4	minimum pseudotrials per class, train	4
4	minimum pseudotrials per class, test	2
4	minimum usable folds	4